



Universidad Nacional de La Matanza

Algoritmos y estructuras de datos

Trabajo práctico:

DoCe



Necesidad

Un equipo de desarrolladores de CodeInc está diseñando un sistema de entretenimiento para un festival. Como parte de la propuesta, se busca incluir un juego simple pero adictivo, en el que los usuarios compitan contra una inteligencia artificial. El juego debe ser dinámico, basado en estrategia y azar, y permitir registrar los resultados para mejorar la IA con el tiempo.

El equipo decidió crear *DoCe*, un juego de cartas por turnos, que simula una batalla entre un jugador humano y una computadora. El objetivo es alcanzar 12 puntos antes que el oponente, utilizando cartas con distintos efectos.

Reglas del juego

- El jugador y la máquina comienzan con 0 puntos y una mano de 3 cartas cada uno.
- Ningún jugador puede tener puntos negativos
- Las cartas tienen diferentes efectos, entre ellos:
 - **+2 o +1 puntos** para quien la juega.
 - **Sacar puntos** al rival (-1 o -2).
 - **Repetir turno** inmediatamente.
 - **Carta Espejo**: permite anular un efecto negativo recibido y devolverlo.
- En cada turno:
 - El jugador elige una carta de su mano y la juega.
 - Roba una nueva carta del mazo al finalizar su jugada.
- Gana el primero que llegue a 12 puntos.
- La máquina puede jugar con distintos niveles de dificultad:
 - **Fácil**: juega cartas al azar.
 - **Medio**: evita jugadas inefectivas:
 - No usa una carta de "sacar puntos" si el oponente tiene 0 puntos.
 - Prioriza cartas que suman puntos si está cerca de ganar (por ejemplo, si tiene 8 o 9 puntos).
 - **Difícil**: elige la mejor jugada disponible según el estado actual del juego.
 - Si el jugador está cerca de ganar, prioriza cartas de "repetir turno" o "sacar puntos".
 - Usa carta espejo en caso de recibir una carta negativa del oponente



- Prefiere repetir turno solo si tiene más de una carta buena en mano

Consigna

Implementar el juego *DoCe* en C con las siguientes características:

Apenas se inicie el programa, deberá haber un menú de 3 opciones:

[A] Jugar

[B] Ver ranking

[C] Salir

Si alguien ingresa a “Jugar”, primero se le pedirá que cargue el nombre de la persona que va a jugar.

Luego, se selecciona el nivel de dificultad contra la máquina:

- Fácil
- Medio
- Difícil

A continuación, se inicia la partida. El jugador humano y la máquina reciben 3 cartas cada uno, y uno de los dos arranca aleatoriamente.

El **mazo de cartas** está compuesto por 40 cartas distribuidas de la siguiente manera:

- 6 cartas **+2 puntos**
- 10 cartas **+1 punto**
- 8 cartas **-1 punto al oponente**
- 6 cartas **-2 puntos al oponente**
- 6 cartas **Repetir turno**
- 4 cartas **Espejo**

Después de jugar una carta, el jugador roba una nueva carta del mazo (si hay disponible). En caso de que se terminen las cartas del mazo y aún nadie haya alcanzado los 12 puntos, se mezclan las cartas descartadas y se continúa jugando con ese nuevo mazo.

Una vez finalizada la partida (cuando uno de los dos jugadores llega a al menos 12 puntos), se generará un **informe** con el detalle de todo lo ocurrido: quién jugó qué carta en cada turno, cuantos puntos acumuló cada jugador y quién ganó la partida. El nombre del archivo debe contener la fecha y la hora actual en el siguiente formato:



informe-juego_YYYY-MM-DD-HH-mm.txt
Por ejemplo: informe-juego_2025-02-01-12-20.txt

Además, se enviará a una API el resultado del juego

```
{
  "codigoGrupo": "ASD123",
  "jugador":
  {
    "nombre": "Juan",
    "vencedor": 0, // o 1 en caso de que haya ganado
  }
}
```

Las configuraciones de la api se leerán de un archivo txt con el siguiente formato:

UrlApi |Codigo identificador grupo

<https://api.com> | ASD123

Por otro lado, deberán subir el código a un repositorio en GitHub. Deberán agregar a su repo:

- Un **readme.md** que indique cómo jugar el juego y qué hacer si quiero cambiar las configuraciones del juego.
- Un **pruebas.md** con diferentes lotes de prueba con el siguiente formato:

Descripción	Salida esperada	Salida obtenida
Se quiere probar qué es lo que pasaría si....	Se espera que....	La salida obtenida fue....

Mínimo se deben documentar 8 casos de prueba, con captura de pantalla de la salida obtenida.



- Un **estrategia.md** que explique las estructuras de datos que decidieron usar, el por qué y que cuente la estrategia utilizada en el juego

La funcionalidad “Ver ranking” muestra por nombre de jugador cuantas partidas ganó cada uno. Para eso, se debe consultar al endpoint ‘<https://algoritmos-api.azurewebsites.net/api/doce/{CodigoGrupo}>’ y mostrar el resultado.

Modalidad de entrega

Se deberá entregar un archivo con el siguiente formato:
TP_ALGORITMOS_2025_1C_{NOMBRE_DE_GRUPO}.zip
Por ejemplo: Si el grupo se llamase “INVISIBLE” -y sus integrantes fueran Spinetta, Pomo, Machi y Gubitsch- el archivo debería llamarse TP_ALGORITMOS_2025_1C_INVISIBLE.zip

Se deberán eliminar las carpetas bin y obj.

En la entrega grupal por miel, se deberá adjuntar el .zip + la URL al repositorio GitHub.

El repositorio debe ser público.

La fecha de entrega del TP es hasta el **04/06**. La defensa del TP se va a realizar entre el 11/06 y el 25/06.

Condiciones básicas para aprobar

- ✓ Código prolijo, dividido en funciones
- ✓ Funciones lo más genéricas posibles
- ✓ Nombres significativos de variables
- ✓ 0 errores 0 warnings
- ✓ Que funcione mínimo para todos los casos de prueba que presentan

Detalle de Endpoints

Esta API permite gestionar rankings de jugadores. Los endpoints disponibles son: **GET**, **POST** y **DELETE**. A continuación, se detallan sus funciones, ejemplos y respuestas.

1. Obtener Rankings (GET)

Descripción:

Obtiene los rankings de jugadores para el código de grupo enviado. Si no hay datos en el grupo, devuelve un array vacío.



Endpoint:

GET <https://algoritmos-api.azurewebsites.net/api/doce/{CodigoGrupo}>

Respuesta:

- **200 OK:** Devuelve un array con los rankings del grupo.
- **400 Bad Request:** Si el grupo no existe o ocurre un error.

Ejemplo de respuesta:

```
[
  {
    "nombreJugador": "giselle",
    "cantidadPartidasGanadas": 10,
  },
  {
    "nombreJugador": "matias",
    "cantidadPartidasGanadas": 2,
  }
]
```

Si el grupo está vacío:

```
[]
```

2. Guardar Rankings (POST)

Descripción:

Guarda jugadores y sus puntajes en el grupo especificado. Si el nombre del jugador ya existe en el grupo, acumula el puntaje.

Endpoint:

POST <https://algoritmos-api.azurewebsites.net/api/doce>

Cuerpo de la solicitud:

```
{
  "CodigoGrupo": "PRUEBA",
  "jugador": {
    "nombre": "Juan",
    "vencedor": 0, // o 1 en caso de que haya ganado
  }
}
```



```
}  
}
```

Respuesta:

- **204 No Content:** Los datos fueron guardados correctamente.
- **400 Bad Request:** Si el grupo no existe o hay algún problema con los datos enviados.

3. Resetear todos los datos (DELETE)

Descripción:

Elimina todos los rankings de un grupo específico, reseteándolo completamente. Este método NO debe estar implementado en su código, es solamente para que puedan limpiar los datos que hayan cargado.

Endpoint:

DELETE <https://algoritmos-api.azurewebsites.net/api/doce/{CodigoGrupo}>

Respuesta:

- **200 OK:** El grupo fue reseteado correctamente.
- **400 Bad Request:** Si el grupo no existe o ocurre un error.